

Optimize Against EA

API Documentation

Developer Reference

GameLab 2 — Julius-Maximilians-Universität Würzburg

Philipp Sehne

`Philipp.Sehne@stud-mail.uni-wuerzburg.de`

David Ahrens

`David.Ahrens@stud-mail.uni-wuerzburg.de`

July 15, 2026

Contents

1	Introduction	3
1.1	Scope	3
1.2	How to read a reference entry	3
1.3	Conventions used in the codebase	3
2	Architecture Overview	4
2.1	Layering	4
2.2	Module map	4
3	Shared Infrastructure	5
3.1	Global state and settings	5
3.2	Layout components	5
3.3	Shared hooks	6
3.4	Hint system	6
3.5	Help system	7
3.6	Firebase	7
3.7	Utilities	8
4	Peak Finder Module (BattleShips)	9
4.1	Type contracts (<code>types/</code>)	9
4.2	Engine (<code>engine/</code>)	11
4.3	EA engine (<code>engine/ea/</code>)	14
4.4	Worker protocol (<code>engine/ea/ea.worker.ts</code>)	16
4.5	Hooks (<code>hooks/</code>)	16
4.6	Key components (<code>components/game/</code>)	17
5	Maze Explorer Module (mazeGame)	19
5.1	Type contracts (<code>types/</code>)	19
5.2	Engine (<code>engine/</code>)	20
5.3	Worker protocol (<code>engine/ea/maze.worker.ts</code>)	24
5.4	Hooks and components	24
6	Shooter Module (shooterGame)	26
6.1	Types and constants (<code>shooter.types.ts</code>)	26
6.2	Core loop (<code>game/core/</code>)	26
6.3	Genetic algorithm (<code>game/ga/</code>)	27
6.4	Stores	27
6.5	Horde mode (<code>horde/</code>)	27
6.6	Mods (<code>mods/</code>)	27
6.7	Hooks and components	28

7	Data Formats	29
7.1	Peak Finder map code (v1)	29
7.2	Maze code	29
7.3	Function problem code	30
7.4	Local storage keys	30
A	Extension Recipes	31
B	Tunable Constants Reference	32

Chapter 1

Introduction

1.1 Scope

Optimize Against EA is a purely client-side React 19 + TypeScript single-page application (built with Vite). It exposes no HTTP/REST endpoints; “API” in this document therefore means the **programmatic interfaces of the codebase**: the exported types, engine functions, Web Worker message protocols, React hooks, and data serialization formats a developer needs in order to extend the application or reuse its parts.

This document complements the *Handbook & User Manual*, which covers the application from the player’s perspective.

1.2 How to read a reference entry

Each documented symbol appears as an entry with its source file in the header, followed by its signature, a description, and where relevant **Params**, **Returns**, **Side effects** and **Notes** fields. All paths are relative to `OptimizeAgainstEA/OptimizeAgainstEAApp/src/`.

1.3 Conventions used in the codebase

- Type-only imports use `import type {...}`.
- React components are **function** declarations; engine and utility functions are **const** arrow functions or plain functions.
- No **any**; all callback parameters are explicitly typed.
- Fitness in the map/maze EAs is *minimized* (0 = optimal); fitness in the shooter GA is *maximized*.
- Randomness is injected as `RNG = () => number` (a `Math.random`-compatible function) so engines stay deterministic under a seeded RNG.

Chapter 2

Architecture Overview

2.1 Layering

Every game module follows the same four-layer structure. Lower layers never import from higher ones:

Layer	Contents
types/ engine/	Pure type contracts and default configurations. No logic. Pure, React-free logic: problem construction, EA operators, codecs, geometry. Unit-testable in isolation.
engine/*.worker.ts	Web Workers running the EA off the main thread; communicate via typed message unions (<code>WorkerInMessage/WorkerOutMessage</code>).
hooks/ components/	React hooks bridging engine/workers to component state. Presentational and interactive React components.

2.2 Module map

Module	Route	Chapter
modules/BattleShips/	/PeakFinder	4
modules/mazeGame/	/MazeExplorer	5
modules/shooterGame/	/ShooterGame, /HordeGame	6
components/, hooks/, context/	— (shared)	3

[TODO: Architecture diagram: modules, workers, stores, and the data flow between them (one figure).]

Chapter 3

Shared Infrastructure

3.1 Global state and settings

SettingsContext / SettingsProvider

context/SettingsContext.tsx

Global settings provider; wraps all routes in `App.tsx`. Holds three independent settings objects, each with its own setter:

```
const { eaSettings,      setEaSettings,      // EASettings
      shooterSettings, setShooterSettings, // ShooterSettings
      hordeSettings,   setHordeSettings,   // HordeSettings
    } = useSettings(); // throws outside a <SettingsProvider>
```

EASettings tunes the Solo-Play shooter GA (mutation rate/strength, presim generations, population size, crossover type, hall of fame, analytics retention, injection deviation). **ShooterSettings** holds starter DNA, round duration, tug-of-war win threshold, player stats and the powerup-choice options. **HordeSettings** is deliberately separate from **EASettings** (own difficulty presets, own starter DNA padded with neutral 0.5s for the Horde-only genes, wave sizing, map id, custom map layout, kills-per-upgrade).

Notes. State is React-only — *not* persisted; a reload restores the `default*Settings`. `resetEASettings()` / `resetShooterSettings()` / `resetHordeSettings()` return fresh defaults (the shooter/horde variants reset the starter DNA to all-0.1 “untrained” genes).

3.2 Layout components

GameLayout, LAYOUT

components/layout/GameLayout.tsx

Three-column page shell (`leftBar | canvas | sidebar`) used by the shooter pages. Exports the `LAYOUT` constant (column widths, spacing, panel colours) for canvas-sizing arithmetic.

Notes. Pass the combined sidebar width to `useScaledCanvas` so the canvas scales correctly.

PageContainer

components/layout/PageContainer.tsx

`PageContainer({ children, backgroundImage? })` — full-viewport page wrapper (`.page-container`); `backgroundImage` is a URL or imported image rendered as a covering, centered background. Default export.

3.3 Shared hooks

useScaledCanvas (worked example)

hooks/useScaledCanvas.ts

```
const { containerRef, scale } = useScaledCanvas({
  baseWidth:    number,    // logical canvas width (e.g. ARENA.WIDTH)
  baseHeight:   number,    // logical canvas height
  sidebarWidth: number,    // total horizontal space taken by side panels
  padding:      number,    // default 16
});
```

Computes the largest uniform scale at which the logical canvas fits the measured container minus sidebarWidth and padding, preserving the aspect ratio.

Returns. containerRef (attach to the element whose size to measure — a ResizeObserver keeps scale current) and the numeric scale factor to apply to the canvas.

useViewport

hooks/useViewport.ts

useViewport(): { W, H } — the current window.innerWidth/innerHeight, re-rendered on every window resize.

useOrientationLock

hooks/useOrientationLock.ts

useOrientationLock(type = 'landscape') — locks the screen orientation while mounted, unlocking on unmount. Only attempts the lock when fullscreen is active (a platform requirement) and re-locks on every fullscreenchange. Fails silently where unsupported (iOS Safari); works on Android Chrome in fullscreen.

useReplayPlayer

hooks/useReplayPlayer.ts

```
const { frameIndex, currentFrame, totalFrames, isPlaying,
  isFirst, isLast, next, prev, goTo, play, pause } =
  useReplayPlayer<Frame>(frames, autoplayIntervalMs = 1200);
```

Playback state machine for any recorded frame list — shared by the PeakFinder EA replay, the Maze EA replay and the generation replay. Generic over the frame type: it only navigates, never looks inside a frame. play(ms?) auto-advances at the given dwell (default autoplayIntervalMs) and self-terminates on the last frame; next/prev/goTo clamp to the valid range.

useSavedLibrary

hooks/useSavedLibrary.ts

```
const { entries, save, remove, rename } =
  useSavedLibrary<Input, Entry extends SavedEntryBase>(storageKey, toEntry);
interface SavedEntryBase { id: string; name?: string; createdAt: number }
```

localStorage-persisted library of player creations — the shared core behind useSavedMaps (bs.savedMaps) and useSavedMazes (maze.savedMazes). toEntry converts the game's input into an entry; its id de-duplicates (saving an existing id is a no-op, and save returns the id either way). A blank name in save/rename clears the name. Storage access is guarded, so private-mode failures degrade to an in-memory list.

3.4 Hint system

Page-wide, toggleable contextual help (components/hints/). Content lives exclusively in hintContent.ts (BattleShips) and modules/mazeGame/hints/mazeHintContent.ts.

useHints (worked example)	components/hints/HintContext.tsx
---------------------------	----------------------------------

```
const { showHint, dismiss } = useHints();
showHint('vsEa.afterProbe', { vars: { gens: '5' }, actions: [...] });
```

Fires a hint by id. No-ops when hints are disabled or the hint is marked *once* and was already seen this session.

Side effects. Persists *enabled* to *localStorage* (*bs.hints.enabled*) and *seen* ids to *sessionStorage* (*bs.hints.seen*).

HintDef	components/hints/hintContent.ts
---------	---------------------------------

```
{ title?, body, style: 'modal' | 'toast', once?, pauses? }
```

pauses: *true* blocks interaction behind a backdrop until dismissed.

HintPopover	components/hints/HintPopover.tsx
-------------	----------------------------------

Anchored, non-blocking coach-mark: wrap any element to pin a hint bubble to it.

```
<HintPopover id="vsEa.replayButton" placement="top" show={canReplay}>
  <button>Watch Last Replay</button>
</HintPopover>
```

Props: *id* (a *HintId* from *hintContent.ts* — the wording lives there), *children* (the anchor), *placement* ('top' | 'top-start' | 'top-end' | 'bottom' | 'left' | 'right', default 'right'), *show* (extra condition on top of *enabled/once-seen*, default *true*), *vars* (fills {placeholders} in the body), *actions* (buttons; falls back to a single "Got it"), *dismissAfter* (auto-dismiss in ms). The bubble renders via a portal to *document.body* and is clamped to stay fully on-screen.

Notes. CSS-grid caveat: the wrapper is *display: inline-block*, so inside a grid it would itself become the grid item — wrap the child *inside* the grid item instead.

3.5 Help system

HELP_TOPICS / HelpButton	components/help/helpContent.tsx
--------------------------	---------------------------------

Content registry for the site-wide help modals. Adding a topic = one entry in *HELP_TOPICS* plus a *<HelpButton topic="..." />*. Each topic provides a *Gameplay* and a *Technical* tab component.

3.6 Firebase

firebase	lib/firebase.ts
----------	-----------------

Exports *firebaseApp* (the initialized Firebase app) and *db* (its Firestore handle). The client-side web config is hardcoded in this file (project *optimizeagainstea*); as a public web API key it is not a secret — access control lives in the Firestore security rules.

Notes. The only consumer is the community Raidboss mode (*modules/shooterGame/game/raidbossStore.ts*); every other feature is fully client-local. Swapping the backing project means editing this one config object.

3.7 Utilities

rng	utils/rng.ts
-----	--------------

```
type RNG = () => number;           // next value in [0, 1)
makeLCG(seed?: number): RNG
```

Seeded linear congruential generator ($s = 1664525 \ s + 1013904223 \bmod 2^{32}$, via `Math.imul` — pure integer math, so the stream is identical on every device). The single random source of the PeakFinder and Maze EAs and of maze generation: same seed $\Rightarrow$ fully reproducible run. Without a seed, a fresh `Math.random()`-derived one is used; the degenerate all-zero state is remapped.

listenable	utils/listenable.ts
------------	---------------------

```
createListenable(): { notify(): void;
                      subscribe(fn: () => void): () => void }
```

The observer plumbing behind every module-level singleton store (`gameStore`, `analyticsStore`, `hordeGameStore`, ...): spread the result into a store object literal; domain methods mutate their fields and call `this.notify()`. `subscribe` returns the matching unsubscribe function, so components can hook in directly or via `useSyncExternalStore`.

Chapter 4

Peak Finder Module (BattleShips)

All paths relative to `modules/BattleShips/`.

4.1 Type contracts (`types/`)

`ProblemInstance` (worked example)

`types/map.ts`

The central abstraction: game modes only ever hold a `ProblemInstance`, never a raw `MapConfig`.

```
interface ProblemInstance {
  evaluate: (x: number, y: number) => number; // 0 = global min, 1 = far away
  bounds: { xMin: number; xMax: number; yMin: number; yMax: number };
  isWin: (x: number, y: number) => boolean;
  displayExponent?: number; // display-only heatmap colour curve; see Notes
  metadata?: { name?: string;
    description?: string;
    globalMinimum?: { x: number; y: number; value: number };
    winRadius?: number }; // extent of the win zone, for overlays
}
```

Notes. Implemented by `createMapProblem` (`engine/functionSurface.ts`) and `createFunctionProblem` (`engine/functionProblem.ts`). New problem sources only need to produce this shape. `evaluate` is the single source of truth — the player’s reading, the EA’s fitness and the heatmap colour all come from it — so a problem must return a value that already spans `[0, 1]` sensibly; do any shaping in the surface engine, never as a display curve. `displayExponent` is the one purely-cosmetic exception: when set, the heatmap colours by `evaluatedisplayExponent` instead of `evaluate` (default 1 = untouched), without ever affecting readings, wins or the EA. Surface maps leave it unset; benchmark functions set it to 0.55 to keep more colour near the optimum.

```

interface Coordinate { x: number; y: number } // normalized 0-1
interface Minimum {
  id: string;
  position: Coordinate;
  isGlobal: boolean; // exactly one minimum per map is the global one
  floor?: number; // depth of a local minimum, as a fraction of basinScale
}
interface MapConfig {
  id: string; // 6-char uppercase share id
  minima: Minimum[];
  bounds: { xMin: number; xMax: number; yMin: number; yMax: number };
  winRadius: number; // normalized radius around the global minimum
  basinScale?: number; // basin width (normalized); see Notes
  createdAt: number; // epoch ms
}
interface ProbeResult { position: Coordinate; value: number; isWin: boolean }

```

Notes. `Minimum.floor` is the local minimum's depth as a fraction of the map's `basinScale` (not an absolute value), so a decoy is equally deceptive on a Small map (wide basins) and a Huge one (narrow basins) — lower = deeper, more deceptive. When omitted, a position-seeded pseudo-random fraction in `[LOCAL_MIN_FLOOR_MIN, LOCAL_MIN_FLOOR_MAX]` = `[0.18, 0.55]` is used, so the same map code always yields the same floors. The floor is ignored (always 0) for the global minimum. `basinScale` is how wide one minimum's basin is in normalized units; it comes from the size preset (`MAP_SIZES`) and fixes a basin's size to the map rather than to how many minima share it. Legacy codes without it fall back to the Medium preset's scale. `bounds` is always the unit square today.

Complete configuration of the Vs-EA opponent. `DEFAULT_EA_CONFIG` provides the defaults (identical to the 'normal' preset); `EA_PRESETS` bundles named presets for the UI.

Field	Default	Meaning
<code>populationSize</code>	30	Individuals per generation
<code>maxGenerations</code>	200	Hard stop
<code>crossoverRate</code>	0.7	Probability of recombination
<code>mutationRate</code>	0.3	Probability of mutating a child
<code>mutationStrength</code>	0.25	Max. perturbation (normalized coords)
<code>mutationDecay</code>	0.90	Per-generation strength multiplier
<code>winPopulationFraction</code>	0.35	Fraction inside win radius $\Rightarrow$ solved
<code>selectionStrategy</code>	'elitist'	'tournament' 'roulette'
<code>crossoverStrategy</code>	'uniform'	'arithmetic' 'singlePoint'
<code>mutationStrategy</code>	'gaussian'	'uniform' 'cauchy'

Individual, Generation, SelectionFn, CrossoverFn, MutationFn, RNG
types/ea.ts

```
interface Individual {
  position: Coordinate;
  fitness: number; // problem.evaluate(position) - minimized, 0 = summit
  isSolution: boolean; // inside the win radius
}
interface Generation {
  index: number;
  individuals: Individual[]; // sorted best-first
  best: Individual;
  meanFitness: number;
}
type RNG = () => number; // Math.random-compatible
type SelectionFn = (population: Individual[], rng: RNG) => Individual;
type CrossoverFn = (a: Individual, b: Individual, rng: RNG) => Coordinate;
type MutationFn = (coord: Coordinate, generationIndex: number,
  config: EAConfig, rng: RNG) => Coordinate;
```

Notes. Operator implementations are looked up from the strategy registries in engine/ea/operators.ts. Mutation receives the generation index so it can apply the decayed strength $\text{mutationStrength} \cdot \text{mutationDecay}^{\text{gen}}$.

4.2 Engine (engine/)

createMapProblem engine/functionSurface.ts

createMapProblem(config: MapConfig): ProblemInstance — builds the smooth surface from the placed minima. First a raw cone surface $s(x, y) = \text{smoothMin}_{m \in \text{minima}}(d((x, y), m) + \text{floor}_m)$ blends the per-minimum cones with a polynomial smooth-minimum (Quilez) so the terrain has no creases along Voronoi seams; floor_m is 0 for the global minimum and the explicit or seeded fractional floor (times **basinScale**) for locals. That raw distance is then mapped to $[0, 1)$ against the map's own **basinScale** by exponential saturation:

$$v(x, y) = 1 - 2^{-b(x, y)^{\text{FAR_FALLOFF}}}, \quad b(x, y) = \frac{s(x, y) - s_{\min}}{\text{basinScale}},$$

where $s_{\min}$ is the deepest minimum centre (so the summit reads 0). One basin away $\Rightarrow b = 1 \Rightarrow v = 0.5$; the map saturates toward (never reaching) 1, so the far field always keeps a usable gradient. Crucially v never consults the surface's own value range, which is why adding minima no longer resizes basins. The mapping is monotone in distance, so it moves no optimum and changes nothing for the EA beyond the scale of the fitness numbers, and deterministic in **config**, so the main thread and the EA worker build identical surfaces. **isWin** tests **isWithinRadius**(point, global, config.winRadius).

Notes. Tunables: **FAR_FALLOFF** (1.3; how steeply open ground climbs once outside a basin), **SMOOTH_K** (0.12; widest smooth-min blend, capped per-map against the tightest minima gap and reduced further if it would sink a local minimum below the global), **LOCAL_MIN_FLOOR_MIN/MAX** (0.18/0.55, as fractions of **basinScale**). Helper exports: **defaultLocalFloor**(position) (the seeded fractional floor an untouched node gets — shown in the create UI) and **effectiveFloor**(minimum, basinScale) (the floor in map units). Legacy maps without a **basinScale** fall back to **FALLBACK_BASIN_SCALE**.

functionProblem

engine/functionProblem.ts

Analytic benchmark surfaces (Sphere, Rastrigin, Eggholder, ...) behind the same ProblemInstance contract as hand-built maps.

```
interface FunctionSpec {           // fully defines a playable surface
  fn: string;                      // benchmark id, or 'procedural'
  cx: number; cy: number;          // where the optimum lands in [0,1]^2
  theta: number;                   // rotation (radians)
  sx: number; sy: number;          // anisotropic scale (zoom)
  rx: 1 | -1; ry: 1 | -1;          // reflections
  seed?: number;                   // procedural surfaces only
}
createFunctionProblem(spec, sharpenOverride?): ProblemInstance
randomFunctionSpec(rng?, { id?, category? }): FunctionSpec
randomSurfaceSpec(): FunctionSpec  // sentinel: re-randomise every load
resolveSpec(spec, rng?): FunctionSpec // sentinel -> concrete procedural
proceduralFunction(seed): BenchmarkFn // seeded fBm "Mystery Surface"
encodeFunctionCode(spec): string / decodeFunctionCode(code): FunctionSpec
```

BENCHMARK_FUNCTIONS holds ~20 classic test functions (several BBOB-derived) grouped into FUNCTION_CATEGORIES (simple | normal | complex | quirky). createFunctionProblem grid-samples the affinely transformed surface once (64×64) to normalise values to [0,1] (85th-percentile ceiling, so corner extremes don't crush the colour range), applies a per-function sharpening exponent, and pins the win target to the lowest sampled point. Winning is value-based: isWin fires once evaluate ≤ WIN_VALUE_EPS (0.04), not at an exact coordinate; metadata.winRadius reports the measured extent of that zone for replay overlays. It also sets displayExponent = 0.55 so the heatmap compresses the highs and keeps more colour near the optimum (a display-only curve — see ProblemInstance); the sharpening exponent above shapes evaluate itself and is separate. Everything is deterministic in the spec (procedural surfaces in seed), so the main thread and the EA worker rebuild identical surfaces.

buildContours

engine/contours.ts

```
function buildContours(
  evaluate: (x: number, y: number) => number, // surface, values in [0,1]
  levels: number[],                          // iso-values to trace
  resolution: number = 80,                   // grid density
): ContourLevel[]
```

Marching-squares isoline extraction. Samples the surface once on a resolution × resolution grid over [0,1]², then emits, per iso-level, the line segments where the surface crosses that level.

Returns. One ContourLevel (level + segments) per requested level; each ContourSegment is x1, y1, x2, y2 in the same normalized [0,1]² space.

geometry helpers

engine/geometry.ts

```
euclideanDistance(a: Coordinate, b: Coordinate): number
isWithinRadius(point: Coordinate, center: Coordinate, radius: number): boolean
closestMinimumDistance(point: Coordinate, minima: Coordinate[]): number
```

Plain 2-D geometry on normalized coordinates. closestMinimumDistance returns Infinity for an empty minima list (used by create mode's spacing check).

sampleGradient / sampleGradientRgb	engine/colorScale.ts
------------------------------------	----------------------

`sampleGradient(t: number): [r, g, b]` — shared colour ramp used by heatmap, contours and charts: a full-spectrum gradient from deep violet ($t = 0$, the summit) through blue, cyan, green, yellow and orange to deep red ($t = 1$, the valley), defined by an easily editable stop list. `sampleGradientRgb(t)` returns the same colour as a CSS `rgb()` string.

valueToHeight	engine/height.ts
---------------	------------------

`valueToHeight(value: number): number` — presentation-only inversion $1 - \text{value}$ (clamped to $[0, 1]$). Internally $0 = \text{summit}$ and $1 = \text{valley}$; the player is shown “height” with $1 = \text{summit}$. Use only for numbers displayed to the player — never feed the result into colour sampling or EA logic.

encodeMap / decodeMap / generateRandomMap	engine/mapCodec.ts
---	--------------------

```
encodeMap(config: MapConfig): string // URL-safe base64 share code
decodeMap(code: string): MapConfig // throws Error('Invalid map code')
generateRandomMap(size: MapSizeId = 'medium'): MapConfig
```

Map share-codes; format specification in Chapter 7. `decodeMap` rejects unknown format versions and any unparsable input with a single `Invalid map code` error; missing `wr/t` fields fall back to `0.04 / Date.now()`, and a missing `bs` to the Medium preset’s `basinScale`. `generateRandomMap` takes a size preset (`MAP_SIZES: 'small' | 'medium' | 'large' | 'huge'`, each fixing a minima count, win radius, min spacing and `basinScale`), places that many minima with the preset’s spacing and promotes a random one to global.

problemCode	engine/problemCode.ts
-------------	-----------------------

```
type ProblemSource = // plain data - survives postMessage
  | { kind: 'map'; config: MapConfig }
  | { kind: 'fn'; spec: FunctionSpec };
decodeProblem(code: string): DecodedProblem // throws on invalid codes
decodeProblemOrNull(code?: string): DecodedProblem | null
buildProblemFromSource(source: ProblemSource): ProblemInstance
```

Uniform entry point for *any* PeakFinder code: function codes carry a `k: 'fn'` marker and are tried first; anything else is treated as a map code. `DecodedProblem` bundles the rebuilt `problem`, a display label (“Map #ABC123” / “f Rastrigin”), the serializable `source` for the EA worker, and `winTarget/winRadius` for overlays. A “random every time” sentinel spec is resolved once here on the main thread, so the worker rebuilds the same concrete surface for the whole session.

codeClipboard	engine/codeClipboard.ts
---------------	-------------------------

`copyCode(text): Promise<void> / pasteCode(): Promise<string>` — clipboard helpers with an in-app fallback: every copy also lands in `localStorage` (`bs.lastCopiedCode`), and paste falls back to that value when the system clipboard is unreadable (Firefox web pages, insecure origins).

4.3 EA engine (engine/ea/)

createEAS stepper (worked example)

engine/ea/evolutionaryAlgorithm.ts

```
function createEAS stepper(
  problem: ProblemInstance,
  config: EAConfig,
  seed?: number,           // seeds the LCG rng - same seed, same run
): EAS stepper

interface EAS stepper {
  step: (n: number) => StepResult; // n = 0 reports without advancing
  currentGenIndex: () => number;
}

type StepResult =
  | { type: 'generation'; generation: Generation; replay?: ReplayFrame[] }
  | { type: 'solved'; generation: Generation; totalGenerations: number;
    replay?: ReplayFrame[] }
  | { type: 'exhausted'; totalGenerations: number; best: Individual };
```

Creates a resumable EA. The generation loop: sort best-first $\rightarrow$ copy elite (top 5%, ≥ 1) $\rightarrow$ select parents $\rightarrow$ crossover (gated by `crossoverRate`) $\rightarrow$ mutate (gated by `mutationRate`) $\rightarrow$ clamp to bounds $\rightarrow$ evaluate $\rightarrow$ win check. The EA counts as solved once $\max(2, \lceil \text{populationSize} \cdot \text{winPopulationFraction} \rceil)$ individuals sit inside the win radius simultaneously.

Notes. The `replay` on each result holds the frames of the breeding that *produced* the reported generation, so the replay's final frame matches that generation's dots. `runEA(problem, config, callbacks, seed?)` is the non-interactive wrapper that steps to completion, reporting through `EACallbacks` (`onGeneration` / `onSolved` / `onExhausted`). The module-level `WIN_POPULATION_FRACTION` (0.10) is only a fallback for configs missing `winPopulationFraction`.

SELECTION_STRATEGIES, CROSSOVER_STRATEGIES, MUTATION_STRATEGIES

engine/ea/operators.ts

Strategy registries mapping each `*Strategy` string to its implementation:

	Strategy	Behaviour
selection	tournament	Draw $k = 3$ random candidates, return the fittest.
	roulette	Fitness-proportionate with inverted weights ($\text{maxFit} - \text{fitness}$), so lower fitness wins more often.
	elitist	Uniform pick from the top 20% only.
crossover	uniform	Each gene (x, y) drawn independently from either parent.
	arithmetic	Blend $\alpha A + (1 - \alpha)B$ with random α .
	singlePoint	One parent contributes x , the other y .
mutation	gaussian	Normally-distributed offset (Box-Muller).
	uniform	Flat offset within $\pm \text{strength}$.
	cauchy	Heavy-tailed offset — occasional long jumps help escape local minima.

All mutation offsets are scaled by the decayed strength `mutationStrength · mutationDecaygen`.

Notes. `CROSSOVER_STRATEGIES_RECORDING` is the single source of truth: those variants additionally return the random choices made (`CrossoverResult` — blend factor α / per-gene source), which the replay log needs and which cannot be reconstructed from the child position once mutation has been applied. The plain `CROSSOVER_STRATEGIES` are thin coordinate-only wrappers that draw identical RNG values.

individual helpers	engine/ea/individual.ts
<pre>createRandom(problem, rng): Individual // uniform in problem.bounds evaluate(position, problem): Individual // wraps fitness + isSolution clamp(coord, problem): Coordinate // clip to problem.bounds</pre>	

eaReplayLog	engine/ea/eaReplayLog.ts
-------------	--------------------------

`buildReplayFrames(before, nextGen, eliteCount, config, solutionThreshold, breeding?)` builds the frame list for one generation; `snapshot(ind, id)` freezes an individual with a stable per-replay id. `ReplayFrame` is a discriminated union on `phase`; every frame carries `headline` and `description` (ready-to-display prose) plus the population as `IndividualSnapshot[]`:

Phase	Extra payload
sorted	population ranked best-first
elite	<code>eliteIds</code> ; <code>nextGen</code> (the elites copied so far)
selection	<code>strategy</code> ; tournament: <code>candidateIds</code> , <code>winnerId</code> ; roulette: normalized <code>weights</code> ; elitist: <code>eliteTopCount</code>
breeding	<code>parentAId/parentBId</code> , <code>child</code> , <code>crossoverResult</code> (pre-mutation position), <code>crossoverStrategy</code> , <code>alpha/geneSource</code> , <code>didCrossover</code>
mutating	<code>childId</code> , <code>didMutate</code> , <code>before/after/delta</code>
newGen	the completed next generation
winCheck	<code>solutionIds</code> , <code>count</code> , <code>threshold</code> , <code>solved</code>

Notes. The breeding/mutating frames show the *real* first non-elite child: a `BreedingRecord` (parents, gate outcomes, pre-mutation position) is captured live inside the stepper’s breeding loop, because those random choices cannot be reconstructed afterwards. The union also declares an `initial` phase, but `buildReplayFrames` never emits it — the stepper hands over an already-sorted population.

4.4 Worker protocol (engine/ea/ea.worker.ts)

WorkerInMessage / WorkerOutMessage (worked example) types/ea.ts

The EA runs in a dedicated Web Worker (engine/ea/ea.worker.ts); one stepper persists in worker scope between messages. The main thread sends WorkerInMessages and receives WorkerOutMessages:

Direction	Type	Payload and meaning
in	START	config: EAConfig, source: ProblemSource — rebuilds the problem and creates the stepper. Does <i>not</i> step, and posts nothing back (generation 0 arrives with the first STEP, avoiding a duplicate).
in	STEP	count: number — advance up to count generations.
in	STOP	Discard the stepper.
out	GENERATION	generation: Generation, replay?: ReplayFrame[] — the replay of the breeding that produced this generation rides along on the same message.
out	SOLVED	generation, totalGenerations, replay?.
out	EXHAUSTED	totalGenerations, best: Individual — maxGenerations reached without solving.
out	ERROR	message: string — e.g. the problem could not be rebuilt from source.

4.5 Hooks (hooks/)

useEARunner (worked example) hooks/useEARunner.ts

```
const ea = useEARunner();
ea.init(source, eaConfig); // create worker, send START - does NOT run
ea.step(n);                // advance n generations (default 1)
ea.stop(); ea.reset();
// state: ea.status, ea.generations, ea.currentGeneration, ea.best,
//        ea.totalGenerations, ea.latestReplay, ea.errorMessage
```

React-side owner of the EA worker. `source` is a serializable `ProblemSource`; `config` defaults to `DEFAULT_EA_CONFIG`. `EASStatus` = `'idle'` | `'running'` | `'solved'` | `'exhausted'` | `'error'`. `latestReplay` holds the `ReplayFrame[]` of the most recent step batch, consumed by the replay overlay.

Notes. Once solved, `totalGenerations` is locked to the solving generation — further probes keep the worker stepping, but no longer advance the “solved in *X* generations” tally. `start` is currently an alias of `init`. Worker errors terminate the worker and surface in `errorMessage`.

replay playback src/hooks/useReplayPlayer.ts

There is no PeakFinder-specific replay hook: `EAReplayOverlay` feeds its `ReplayFrame[]` to the shared, frame-agnostic `useReplayPlayer<Frame>(frames, autoplayIntervalMs?)`, which provides `currentFrame`, `frameIndex`, `totalFrames`, `isFirst/isLast`, `next/prev/goTo` and `play/pause` autoplay.

useGameMaphooks/useGameMap.ts

Create-mode map state.

```
const map = useGameMap(maxMinima?);  
// state: minima, mapId, maxMinima, minSpacing, isFull, hasGlobal,  
//        globalMinimum  
// actions: addMinimum(pos), removeMinimum(id), setGlobalMinimum(id),  
//           setFloor(id, floor | undefined), clearAll(),  
//           getMapConfig(), getCode()
```

`addMinimum` silently rejects placements beyond `MAX_MINIMA` (12) or closer than `MIN_SPACING` (0.12) to an existing minimum. `setFloor(id, undefined)` reverts a node to its random default depth. `clearAll` also mints a fresh `mapId`, so each created map saves as its own entry. `getCode() = encodeMap(getMapConfig());` the win radius is fixed at 0.04.

usePlaySessionhooks/usePlaySession.ts

```
const { probes, status, bestProbe, probe, reset } =  
  usePlaySession(problem: ProblemInstance | null);
```

Play-mode session state. `probe(position)` evaluates the problem and appends a `ProbeResult`; `status` is `'idle' | 'playing' | 'won'` and stays `'won'` once won (probing remains possible for post-win exploration). `bestProbe` is the probe with the lowest value so far.

useSavedMapshooks/useSavedMaps.ts

```
const { savedMaps, saveMap, removeMap, renameMap } = useSavedMaps();  
interface SavedMap { id; name?; code; createdAt; minimaCount }
```

The player's saved-map library — a thin domain wrapper around the shared `useSavedLibrary` hook, persisted in `localStorage` under `bs.savedMaps`. `saveMap(config: MapConfig)` stores the encoded share code; saving an id that already exists is a no-op.

4.6 Key components (components/game/)

GameMap (worked example)components/game/shared/GameMap.tsx

The central square map canvas (grid + heatmap + minima dots + overlays).

```
minima?: Minimum[]; showMinima?: boolean; highlightGlobal?: boolean;  
evaluateFn?: (x, y) => number; // enables the HeatmapLayer  
heatmapConfig?: Partial<HeatmapConfig>;  
revealPoints?: Coordinate[]; // heatmap only shown near these  
exclusionRadius?: number; // dashed rings around minima (create mode)  
onMapClick?: (coord: Coordinate) => void; // normalized coords  
onMinimumClick?: (id: string) => void; selectedId?: string | null;  
overlayLabel?: string; className?: string; children?: ReactNode;
```

Clicks are converted to normalized $[0,1]^2$ coordinates; clicks on a minimum dot go to `onMinimumClick` instead of `onMapClick`. Probe markers, win rings etc. are passed as absolutely-positioned children.

ContourLayer / HeatmapLayer	components/game/shared/
-----------------------------	-------------------------

Both render a surface visualization from `{ evaluate, config?, revealPoints? }`. Reveal semantics: `undefined` reveals the whole map; an array (even an empty one) reveals only circles of `revealRadius` around its points. `GameMap` currently renders the heatmap; `ContourLayer` (SVG marching-squares isolines, levels packed densely near the summit) remains available for contour-style views. Defaults: `DEFAULT_CONTOUR_CONFIG = { lineCount: 24, spacingExponent: 2.5, resolution: 100, revealRadius: 0.1 }`; `DEFAULT_HEATMAP_CONFIG = { resolution: 1080, opacity: 0.82, revealRadius: 0.05, valueExponent: 1 }`. `valueExponent` is the display curve applied as `valuevalueExponent` before colouring; 1 sends the value straight to the ramp (what surface maps use), while callers pass a problem's `displayExponent` through it (benchmark functions use 0.55). It only affects colour — never readings, wins or the EA.

EAREplayOverlay / ReplayMap / IndividualList	components/game/vs-ea/
--	------------------------

Full-screen replay modal: `EAREplayOverlay({ frames, onClose })` drives the shared `useReplayPlayer` through the frame list. `ReplayMap` renders the population dots plus per-phase decorations: `highlightIds/dimIds/markerIds/solutionIds` sets, `customOpacities` (roulette weights), `extraDots` (the bred child), `arrow` (mutation offset), `parentLine` (arithmetic crossover) and `geneLines` (uniform/single-point gene sources). Dots glide with `DOT_MOVE_DURATION = 1000ms`. `IndividualList` lists the population with role badges (ELITE, PARENT A/B, CHILD, WIN, WINNER, CANDIDATE, ELIGIBLE) via roles: `Map<id, IndividualRole>` and optional per-row annotations.

FitnessChart, ModeSelector, CodeModal	components/game/shared/
---------------------------------------	-------------------------

`FitnessChart({ series, yMax?, compact?, yLabel?, hoveredIndex?, onHover?, markers? })` — SVG line chart of probe/EA values over time (y-axis shown as the player-facing “height”). `ModeSelector({ onSelect })` — the Create / Play / Vs EA landing picker. `CodeModal({ code, mapId, onClose, onPlayNow? })` — share-code dialog with clipboard copy via `copyCode`.

SavedMapsSidebar / SavedFunctionsSidebar	components/game/shared/
--	-------------------------

Two self-contained left-edge drawers (no props). `SavedMapsSidebar` lists the player's saved maps (`useSavedMaps`) with rename/delete; clicking an entry copies its share code. `SavedFunctionsSidebar` lists the analytic benchmark functions grouped by category; copying a function mints a *fresh* random transform each time, and “Random every time” copies the sentinel spec that re-randomises the whole surface on every load.

VsEAMode and overlays	components/game/vs-ea/
-----------------------	------------------------

`VsEAMode` wires the race: it owns the `EACConfig` (initialised to the ‘normal’ preset), steps the EA per player probe, and mounts `EASettingsPanel` (difficulty presets from `EA_PRESETS` plus advanced sliders), `EAWinOverlay / SecondSolveOverlay` (EA won / player beat its solution on the retry) and `GenerationReplayOverlay` (dot movement across whole generations, as opposed to the phase-by-phase `EAREplayOverlay`).

Chapter 5

Maze Explorer Module (mazeGame)

All paths relative to `modules/mazeGame/`. The module mirrors the `BattleShips` structure with a *discrete* genome: an individual is a fixed-length move sequence instead of a point.

5.1 Type contracts (`types/`)

Maze core types (worked example)

`types/maze.ts`

```
interface Cell { x: number; y: number } // grid coordinates
type Move = 0 | 1 | 2 | 3; // indexes into MOVE_ARROWS / MOVE_DELTAS
type Path = Move[]; // the genome: fixed-length move sequence
const MOVE_ARROWS = ['↑', '→', '↓', '←'];
type WallRule = 'waste' | 'break' | 'repair';
type FitnessFnId = 'manhattan' | 'geodesic' | 'length' | 'novelty';

interface Grid { cols: number; rows: number; walls: Uint8Array }
interface SerializedMaze extends Grid { start: Cell; goal: Cell }
interface WalkResult { // the phenotype of a Path
  visited: Set<number>; // visited cell indices (y*cols + x)
  trail: Cell[]; // cell per step; length = path.length + 1
  finalCell: number; // where the agent settled
  reachedGoalAt: number; // step it first stood on the goal, or -1
  crashedAt: number; // wall-crash step ('break' rule only), or -1
}
interface MazeProblem {
  grid: Grid; cols; rows; start: Cell; goal: Cell;
  geodesicField: Int32Array; // BFS distance to goal; -1 = unreachable
  pathLength: number; // auto-sized genome length L
  fitnessFnId: FitnessFnId; wallRule: WallRule;
  evaluate: (walk: WalkResult) => number; // lower = better, 0 = optimal
  isWin: (walk: WalkResult) => boolean;
  metadata: { seed: number; shortestPath: number };
}
```

Notes. `Grid.walls[i]` holds an *open-passage* bitmask per cell $i = y \cdot \text{cols} + x$ ($N=1$, $E=2$, $S=4$, $W=8$; a set bit means the passage is open — `MOVE_WALL_BIT`); `cellIndex(x, y, cols)` maps 2-D coordinates to flat indices. `SerializedMaze` decouples a hand-built creator maze from procedural generation (start/goal are arbitrary). `MazeProblem` parallels `BattleShips`' `ProblemInstance`, but `evaluate`/`isWin` take a simulated `WalkResult` instead of an (x, y) coordinate. `WallRule` is the constraint-handling strategy for blocked moves: **waste** loses the move, **break** crashes and freezes the agent (death penalty), **repair** rewrites the gene to a random open direction and takes it (Lamarckian).

Discrete-EA configuration. Shape mirrors the BattleShips EAConfig, plus three maze-only fields (fitnessFnId, wallRule, pathLengthFactor); an Individual carries a path and its cached walk instead of a position.

Field	Default	Meaning
populationSize	100	Individuals per generation
maxGenerations	300	Hard stop
crossoverRate	0.85	Probability of recombination
mutationRate	0.95	Probability of mutating a child
mutationStrength	0.06	Per-gene re-pick <i>probability</i> (not a perturbation magnitude)
mutationDecay	1.0	Per-generation strength multiplier (no decay — keeps exploration alive)
winPopulationFraction	0.08	Fraction that must reach the goal $\Rightarrow$ solved
selectionStrategy	'tournament'	'roulette' 'elitist'
crossoverStrategy	'singlePoint'	'uniform'
mutationStrategy	'point'	'segment'
fitnessFnId	'geodesic'	Which fitness function the problem evaluates against
wallRule	'waste'	Blocked-move handling ('break' 'repair')
pathLengthFactor	3.5	Genome length as a multiple of the shortest path (≥ 1 ; baked in at build time — changing it restarts the run)

5.2 Engine (engine/)

```
generateMaze(cols, rows, seed, { braid?, openness? }): Grid
pickRandomStartGoal(grid, rng, minFrac = 0.6): { start; goal }
gridFromEdgeWalls(cols, rows, isHWall, isVWall): Grid
```

Seeded procedural generation — identical seed $\Rightarrow$ identical maze (the basis of the fitness-comparison experiment). **generateMaze** runs an iterative recursive backtracker (every cell reachable), then two seeded post-passes that only ever *remove* walls, so connectivity is preserved: **braid** opens a fraction of dead-ends into loops, **openness** removes a fraction of the remaining interior walls (punching side doors into the backtracker's long corridors, creating parallel routes the population can split across).

Notes. **pickRandomStartGoal** places the goal anywhere and draws the start from cells whose corridor distance to the goal is at least **minFrac** of the maze's BFS diameter — never trivially close. **gridFromEdgeWalls** converts the maze creator's edge-wall model into a **Grid**. **mazeToAscii** is a debug-only ASCII renderer.

<code>geodesic</code>	<code>engine/geodesic.ts</code>
-----------------------	---------------------------------

```
computeGeodesic(grid: Grid, goal: Cell, start: Cell): GeodesicResult
interface GeodesicResult {
  field: Int32Array;           // corridor distance to goal; -1 = unreachable
  diameter: number;           // largest finite distance in the field
  shortestFromStart: number;  // start -> goal distance
}
```

BFS distance field flooded outward from the goal through open corridors, computed once at problem build time. Backs the `geodesic` and `length` fitness functions, genome auto-sizing, and `pickRandomStartGoal`.

<code>mazeProblem</code>	<code>engine/mazeProblem.ts</code>
--------------------------	------------------------------------

`createMazeProblem(opts: BuildMazeOptions): MazeProblem` — generates the maze (or accepts an explicit `grid` for creator mazes), floods the geodesic field, auto-sizes the genome length $L = \min(\text{MAX_PATH_LENGTH}, \lceil \text{shortest} \cdot \text{pathLengthFactor} \rceil)$, and wires up the chosen fitness function. Changing only `fitnessFnId` for the same seed yields the *same* maze with a different `evaluate` — the comparison demo. All fitness functions are “lower = better”, normalized to roughly $[0, 1]$, and score the agent’s *final* cell (the goal absorbs), so a lucky fly-through earns no credit:

Id	Scoring
<code>manhattan</code>	Straight-line distance of the final cell to the goal — ignores walls, so it is <i>deceptive</i> : the population piles up in the dead-end nearest the goal as the crow flies, and stalls.
<code>geodesic</code>	Corridor (BFS) distance of the final cell to the goal, normalized by the maze diameter — smooth and solvable.
<code>length</code>	Geodesic plus a reward for reaching the goal early: non-solvers score in $[0.5, 1]$, solvers in $[0, 0.5)$ scaled by <code>reachedGoalAt</code> — reaching dominates, shorter wins among solvers.
<code>novelty</code>	Placeholder (constant 1): novelty depends on the whole population + archive, so the stepper applies a <code>NoveltyScorer</code> that overwrites each individual’s fitness.

Notes. `isWin` is `reachedGoalAt >= 0` regardless of fitness function. Defaults: `DEFAULT_BRAID` = 0.5, `DEFAULT_OPENNESS` = 0.25 (too high and Manhattan stops being deceptive), `MAX_PATH_LENGTH` = 600 (exported so the creator can warn when a maze’s shortest path exceeds what a genome can hold).

createNoveltyScorer (worked example)	engine/ea/novelty.ts
--------------------------------------	----------------------

```
function createNoveltyScorer(
  cols: number, rows: number, k: number = DEFAULT_K,
): NoveltyScorer
```

Novelty-search infrastructure — the canonical maze-deception demo (Lehman & Stanley): objective-chasing gets trapped, pure exploration floods the maze and stumbles onto the exit. Instead of rewarding closeness to the goal, each individual is rewarded for ending up somewhere *few* others (now or in the past) have ended up. Used when `FitnessFnId = 'novelty'`; because it depends on the whole population plus an archive, the stepper calls `scorer.score(pop)` each generation instead of a per-individual `evaluate`.

Notes. Behaviour descriptor = the final cell the agent settles on. `score` sets each individual's fitness to $1 - \min(1, \text{novelty}/\text{maxDist})$, where novelty is the mean Euclidean distance to the k nearest behaviours ($k = 15$ by default, own behaviour excluded) among the current population plus the archive. Archive admission: every *newly explored* distinct final cell is added (deduplicated by cell index); `archiveSize()` exposes the count for visualization.

individual helpers / walkPath	engine/ea/individual.ts
-------------------------------	-------------------------

```
walkPath(problem, path, rng?): WalkResult    // simulate the phenotype
evaluate(path, problem, rng?): Individual    // walk + score + isSolution
createRandom(problem, rng?): Individual      // random genome of length L
repair(path, problem, rng?): Path            // enforce genome length L
randomMove(rng?): Move
```

`walkPath` executes the genome move by move. The trail is always `path.length + 1` cells (uniform for animation): a blocked move keeps the agent in place under `waste`, freezes it for the rest of the walk under `break` (recorded in `crashedAt`), or rewrites the gene to a random open direction under `repair` (Lamarckian — `evaluate` therefore copies the genome first, so shared parent arrays are never mutated). The goal *absorbs*: once reached, the agent stays there, which is what makes the final cell a meaningful fitness target. `repair` is the length-invariant safety net (truncate / pad with random moves).

SELECTION_STRATEGIES, CROSSOVER_STRATEGIES_RECORDING, MUTATION_STRATEGIES	engine/ea/operators.ts
---	------------------------

Selection is ported verbatim from BattleShips (tournament $k = 3$ / roulette with inverted weights / elitist top 20% — it only reads `.fitness`). Crossover and mutation work on the move sequence:

	Strategy	Behaviour
crossover	<code>singlePoint</code>	Follow parent A up to a random cut ($1..L-1$), then switch to parent B; records cut.
	<code>uniform</code>	Each gene independently drawn from either parent; records the per-gene <code>geneMask</code> .
mutation	<code>point</code>	Each gene independently re-picked with probability $\text{mutationStrength} \cdot \text{mutationDecay}^{\text{gen}}$.
	<code>segment</code>	Re-randomize one contiguous block whose max length scales with the decayed strength (a “kink” the path forks at).

Notes. Both operator families are recording variants (`CrossoverResult` / `MutationResult` with `mutatedIndices`) — the replay log needs the splice cut / gene mask / mutated positions, which cannot be reverse-engineered from the child genome.

createMazeEAS stepper**engine/ea/evolutionaryAlgorithm.ts**

createMazeEAS stepper(problem, config, seed?): **EAS stepper** — same resumable-stepper contract and **StepResult** union as **BattleShips** (Chapter 4), with two maze-specific additions:

- **updateConfig**(nextConfig, nextProblem?) — swap in new tuning *without* discarding the population. When **nextProblem** is given (fitness function or wall rule changed), every genome is kept but re-walked and re-scored under the new problem.
- Novelty runs use a solution threshold of 1 (a single explorer finding the exit counts as solved) — novelty spreads the population out, so it never piles a fraction onto the goal. All other fitness functions use $\max(2, \lceil \text{populationSize} \cdot \text{winPopulationFraction} \rceil)$.

Elitism is top 5% (≥ 1), as in **BattleShips**.

eaReplayLog**engine/ea/eaReplayLog.ts**

Same phase set and frame structure as the **BattleShips** replay log (**sorted** → **elite** → **selection** → **breeding** → **mutating** → **newGen** → **winCheck**), re-based on paths: **IndividualSnapshot** carries **path**, **trail** and **finalCell** instead of a position (the map draws trails as spaghetti, the list renders genomes as arrow strings). Payload differences: **breeding** records **cut/geneMask** instead of $\alpha/\text{geneSource}$; **mutating** records **beforePath/afterPath/mutatedIndices** instead of a coordinate delta.

mazeCodec**engine/mazeCodec.ts**

```
mazeId(maze: SerializedMaze): string    // content-derived 6-char id
encodeMaze(maze: SerializedMaze): string
decodeMaze(code: string): SerializedMaze // throws Error('Invalid maze code')
```

Maze share-codes; format in Chapter 7. **mazeId** hashes the maze content (djb2 → base36), so identical mazes get identical ids — what the saved-maze library de-duplicates on. **decodeMaze** validates version, dimensions (2..64 per axis), wall string length and marker bounds, and throws a single **Invalid maze code** error otherwise.

5.3 Worker protocol (`engine/ea/maze.worker.ts`)

WorkerInMessage / WorkerOutMessage

`types/ea.ts`

Same shape as the BattleShips protocol; the out-messages (`GENERATION` / `SOLVED` / `EXHAUSTED` / `ERROR`) are identical apart from the maze-typed payloads. Differences on the in-side:

Direction	Type	Payload and meaning
in	START	<code>config: EAConfig</code> , <code>seed</code> , <code>cols</code> , <code>rows</code> , <code>maze?: SerializedMaze</code> — a hand-built creator maze takes precedence; otherwise the maze is generated procedurally from <code>seed</code> . The EA RNG is seeded with the same value, so on the same seed the <i>only</i> thing that varies between fitness functions is <code>evaluate</code> — the comparison demo.
in	STEP	<code>count: number</code> .
in	UPDATE_CONFIG	<code>config: EAConfig</code> — retune the running EA without restarting; takes effect on the next generation. Only a changed <code>fitnessFnId</code> or <code>wallRule</code> rebuilds the problem (and re-scores the population); everything else is pure tuning.
in	STOP	Discard the stepper.

Notes. `pathLengthFactor` is fixed for the worker's lifetime: the main thread sends a fresh `START` when it changes, so mixed-length populations can never occur.

5.4 Hooks and components

useMazeEARunner

`hooks/useMazeEARunner.ts`

```
const ea = useMazeEARunner();
ea.init({ seed, cols, rows, maze?, pathLengthFactor? }, config?);
ea.step(n); ea.updateConfig(config); ea.stop(); ea.reset();
// state: status, generations, currentGeneration, best,
//         totalGenerations, latestReplay, errorMessage
```

Mirrors BattleShips' `useEARunner` (same `EAState`, same solved-lock on `totalGenerations`, `start` again an alias of `init`); the differences are the `START` payload (`MazeRunParams` instead of a `ProblemSource`), the default config (`DEFAULT_MAZE_EA_CONFIG`) and the extra `updateConfig` for live retuning. Replay playback again uses the shared `useReplayPlayer` — there is no maze-specific replay hook.

useSavedMazes

`hooks/useSavedMazes.ts`

```
const { savedMazes, saveMaze, removeMaze, renameMaze } = useSavedMazes();
interface SavedMaze { id; name?; code; createdAt; cols; rows }
```

Saved-maze library over the shared `useSavedLibrary`, persisted in `localStorage` under `maze.savedMazes`. Entries are keyed by the content-derived `mazeId`, so saving an identical maze is a no-op; `saveMaze` returns the id either way.

MazeCanvas	components/shared/MazeCanvas.tsx
The shared maze renderer (walls, start/goal markers, overlays). Key props: <code>cols/rows/walls/start/goal</code> plus <code>trails</code> (population / player paths, drawn in order), <code>solidCells</code> (creator's painted blocks), <code>markers</code> (highlight dots, e.g. splice / mutated cells), <code>agents</code> (animated dots drawn above walls), <code>previewWall</code> (creator hover ghost), <code>showGrid</code>, and a fog-of-war cell set used by Solve mode to reveal the maze as it is explored.	
MazeSetupScreen / MazeExperimentMode / MazeEASettingsPanel	components/experiment/
MazeSetupScreen — experiment entry: pick a fresh random maze (size / openness of your choosing) or a saved creation, then start. MazeExperimentMode — the run itself: owns the EACConfig, drives useMazeEARunner, and pins settings changes to the generation where they took effect as markers on the fitness chart. MazeEASettingsPanel(<code>{ config, onConfigChange, shortestPath?, runStarted?, onClose }</code>) — the EACConfig editors (also exported chrome-free as MazeEASettingsControls); the genome-length slider is locked once a run has started, since length is baked into every genome.	
MazeEAReplayOverlay / MazePathMap / MazeIndividualList	components/experiment/replay/
Full-screen dissection of one generation — a re-skin of BattleShips' replay overlay with an identical phase set. MazeEAReplayOverlay(<code>{ frames, problem, onClose }</code>) steps the shared useReplayPlayer. MazePathMap draws every individual's walked trail over the maze (highlight/dim/marker/solution id-sets, <code>customOpacities</code> for roulette, <code>selectedId</code> to lift one path above the rest, splice-point cell markers). MazeIndividualList renders genomes as arrow strings with role badges, per-gene highlights (crossover source / mutated genes), <code>maxGenes</code> truncation and click-to-select (<code>onSelect</code>).	
MazeCreateMode / MazePlayMode	components/create/, components/play/
MazeCreateMode — the maze editor: draw/erase walls on cell edges (elliptical edge hitboxes so free-form strokes don't grab stray orthogonal walls), place start/goal, size 4–32 per axis, export via <code>encodeMaze</code>. MazePlayMode — the player writes a move string with a D-pad and watches it walk (one step per 140 ms tick), edits it, and re-submits — experiencing genome, phenotype and mutation by hand; a ghost replays the previous attempt for comparison.	

Chapter 6

Shooter Module (shooterGame)

All paths relative to `modules/shooterGame/`.

6.1 Types and constants (`shooter.types.ts`)

DNA (worked example)	<code>shooter.types.ts</code>
----------------------	-------------------------------

An agent genome is `number[]` of length 7; all genes in $[0, 1]$. Indices via `DNA_INDEX`:

#	Gene	Effect
0	AGGRESSION	Chase strength
1	DODGE_WEIGHT	Bullet-avoidance strength
2	SHOOT_ACCURACY	Aim precision (1 = perfect)
3	PREFERRED_RANGE	Combat distance ($\times 300\text{px} + 100\text{px}$)
4	MOVEMENT_SPEED	Speed bonus ($40\text{ base} + \text{gene} \times 80\text{px/s}$)
5	PREDICT_LEAD	Aim lead factor
6	FIRE_RATE	Fire rate (higher = faster)

Related constants: `ARENA` (800×800 logical px), `GAME_CONFIG` (round duration, bullet speed, cooldowns), `STARTER_DNA`. `GamePhase` = `'idle'` | `'playing'` | `'roundEnd'` | `'evolving'`.

6.2 Core loop (`game/core/`)

<code>gameLoop.update</code>	<code>game/core/gameLoop.ts</code>
------------------------------	------------------------------------

Pure function `update(state, dt, input) → GameState`; no React dependencies. Also accumulates `PlayerGhostFrame[]` into `GameState.ghostFrames` for ghost-based evolution. **[TODO: Full signature and state transitions.]**

<code>renderer</code>	<code>game/core/renderer.ts</code>
-----------------------	------------------------------------

Canvas draw calls. **[TODO: Entry.]**

<code>vec</code>	<code>game/core/vec.ts</code>
------------------	-------------------------------

2-D vector math: `add`, `sub`, `scale`, `normalize`, `angle`, `clamp`, `perpendicular`, ... **[TODO: Signatures.]**

6.3 Genetic algorithm (`game/ga/`)

<code>evolve</code> (worked example)	<code>game/ga/evolution.ts</code>
--------------------------------------	-----------------------------------

Produces the next generation after a round: tournament selection → single-point crossover → per-gene mutation, driven by each agent's round fitness. Companions:

- `getNextAgent(population): number[]` — DNA of the next opponent to spawn;
- `presimulate(generations): Population` — round-robin pre-training (every agent vs. every other) before the first round;
- `presimulateAgainstGhost(generations, ghost)` — evolves the population against a recording of the player's previous round.

[TODO: Exact signatures and config knobs.]

<code>fitness</code>	<code>game/ga/fitness.ts</code>
----------------------	---------------------------------

`calculateFitness(stats: RoundStats): number`, `calculateRaidbossFitness(stats, roundDuration)`, `normalizeFitness(fitnesses)`. **[TODO: Scoring formulas.]**

<code>population</code>	<code>game/ga/population.ts</code>
-------------------------	------------------------------------

`initPopulation`, `updatePopulationStats`. **[TODO: Entry.]**

<code>evolution.worker.ts</code>	<code>game/ga/evolution.worker.ts</code>
----------------------------------	--

[TODO: Worker message protocol for off-thread evolution/presimulation.]

6.4 Stores

The shooter uses custom observable stores (not Zustand/Redux): components subscribe via `store.subscribe(cb)` and read `store.state` directly.

<code>gameStore</code>	<code>game/gameStore.ts</code>
------------------------	--------------------------------

Holds `GameState` outside React; consumed by `ShooterCanvas` and `DNADisplay`. **[TODO: State shape and mutation API.]**

<code>raidbossStore</code> / <code>analyticsStore</code>	<code>game/</code>
--	--------------------

[TODO: Entries; document the Firebase interaction of the raidboss store.]

6.5 Horde mode (`horde/`)

<code>horde subsystem</code>	<code>horde/</code>
------------------------------	---------------------

`hordeTypes.ts`, `hordeDna.ts`, `hordeMaps.ts`, `hordePathfinding.ts`, `hordeCollision.ts`, `hordeGameStore.ts`, `hordeRunStore.ts`. **[TODO: One entry per file: map format, pathfinding algorithm, wave/DNA evolution, store shapes.]**

6.6 Mods (`mods/`)

<code>mods subsystem</code>	<code>mods/</code>
-----------------------------	--------------------

`modTypes.ts`, `runModsStore.ts`, `shotEngine.ts`. **[TODO: Mod type registry and how mods alter the shot engine.]**

6.7 Hooks and components

useGameLoop / useInput / useTouchControls

hooks/

useGameLoop: requestAnimationFrame loop calling update() and driving gameStore. useInput: keyboard + mouse → InputState. useTouchControls: virtual joystick / aim zone. **[TODO: Signatures.]**

ShooterCanvas, HordeCanvas, DNADisplay, MobileJoystickZone, MobileAimZone
components/

[TODO: One short entry each.]

Chapter 7

Data Formats

Serialization formats are the closest thing the app has to a public API: they cross device boundaries via share codes and local storage.

7.1 Peak Finder map code (v1)

Produced by `encodeMap` (`BattleShips/engine/mapCodec.ts`): a URL-safe Base64 encoding of

```
{ "v": 1,                // format version
  "id": "ABC123",         // map id
  "m": [[x, y, isGlobal, floor?], ...], // minima (normalized coords)
  "wr": 0.04,             // win radius (fraction of map width)
  "bs": 0.12,            // basinScale (optional; see below)
  "t": 1699999999999 }   // timestamp
```

The optional 4th tuple entry (`floor`) is only present for local minima with an explicit depth — omitting it keeps older codes valid and round-trips untouched (random-floor) nodes unchanged; the stored floor is a fraction of `basinScale`. The optional `bs` (basin scale) is written only when present on the config; older codes without it decode with the Medium preset's `basinScale`, so they stay valid. Coordinates are rounded to 4 decimals. `decodeMap` throws a single `Error('Invalid map code')` for anything unparsable or with an unknown `v`; a missing `wr` defaults to 0.04 and a missing `t` to `Date.now()`.

7.2 Maze code

Produced by `encodeMaze` in `mazeGame/engine/mazeCodec.ts`: a URL-safe Base64 encoding of

```
{ "v": 1,                // format version
  "id": "ABC123",         // content-derived id (djb2 hash, base36)
  "c": cols, "r": rows,
  "s": [x, y],           // start cell
  "g": [x, y],           // goal cell
  "w": "af3...",         // one hex digit per cell: open-passage nibble
                          // (N=1 E=2 S=4 W=8), row-major
  "t": 1699999999999 } // timestamp
```

The full wall grid is encoded (not a generator seed), so hand-built creator mazes round-trip exactly; 32×32 — the creator's maximum — is 1024 hex digits, comfortably copy-pasteable. `decodeMaze` validates version, dimensions (2.64), wall-string length and marker bounds, throwing `Error('Invalid maze code')` otherwise.

7.3 Function problem code

Produced by `encodeFunctionCode` in `BattleShips/engine/functionProblem.ts`: a URL-safe Base64 encoding of

```
{ "k": "fn",           // marker - distinguishes from map codes
  "fn": "rastrigin",   // benchmark id, "procedural", or "random"
  "c": [cx, cy],       // optimum position in the [0,1] viewport
  "th": theta,         // rotation (radians)
  "s": [sx, sy],       // anisotropic scale
  "r": [rx, ry],       // reflections (1 | -1)
  "sd": seed }         // only for procedural surfaces
```

`decodeProblem` (`BattleShips/engine/problemCode.ts`) tries the function decoder first (keyed on the `k:"fn"` marker), then falls back to map decoding. A code with `fn:"random"` is a sentinel that resolves to a fresh procedural surface on every load.

7.4 Local storage keys

Key	Contents
<code>bs.hints.enabled</code>	Hint system on/off (<code>localStorage</code>)
<code>bs.hints.seen</code>	Seen hint ids (<code>sessionStorage</code> , per session)
<code>bs.savedMaps</code>	Saved Peak Finder maps (<code>useSavedMaps</code>)
<code>bs.lastCopiedCode</code>	In-app clipboard fallback for share codes
<code>maze.savedMazes</code>	Saved mazes (<code>useSavedMazes</code>)

Appendix A

Extension Recipes

Short how-tos for the most likely extension points. **[TODO: Flesh out each recipe with concrete steps.]**

- **Add a new analytic problem to Peak Finder** — implement a `ProblemInstance` via `createFunctionProblem` and register it.
- **Add a new EA operator** — add an entry to the relevant `*_STRATEGIES` registry and extend the strategy union type.
- **Add a new maze fitness function** — extend `FitnessFnId`, `FITNESS_FN_IDS` and `FITNESS_FN_LABELS`, implement the scorer in `mazeProblem.ts`.
- **Add a help topic** — one entry in `HELP_TOPICS` + `<HelpButton topic="..." />`.
- **Add a hint** — one entry in `hintContent.ts`, fire via `showHint(id)` or wrap with `<HintPopover>`.
- **Add a new mini-game module** — follow the four-layer module structure (Chapter 2) and register a route in `App.tsx`.

Appendix B

Tunable Constants Reference

Central table of gameplay/visual constants and their locations. **[TODO: Import the full tunables table (currently maintained in `claude.md`) and keep the two in sync — or declare one canonical.]**